Forecast-Aware Rolling-Horizon Optimization for Hybrid Wind-Storage Dispatch Under Uncertainty

David Valenta
North Carolina State University, Raleigh, NC, USA
Research advised by Dr. Chris Qin, Washington State University, Vancouver, WA, USA

Main claim: Hybrid wind-storage value depends on the whole information chain: forecast accuracy, replanning horizon, uncertainty handling, and physically valid storage constraints. The best tested deployable case was a three-scenario 48-hour controller, which reduced COVE by 23.19% and increased revenue by 30.19% relative to baseload on the frozen 2014-2023 test period.

Abstract

Hybrid wind farms can either sell wind energy immediately or use storage to shift energy toward more valuable hours. This decision is difficult because both wind generation and electricity price are uncertain. A storage controller that looks too little into the future may miss high-price opportunities, but a controller that trusts a bad long-range forecast can make worse decisions than doing nothing. This paper presents a forecast-aware rolling-horizon optimization pipeline for a Pyron wind farm case study with compressed-air energy storage. The pipeline first compares several wind-power forecasting methods, then feeds the selected causal ridge forecast into a mixed-integer linear programming dispatch model solved with Gurobi. The dispatch model includes wind-only charging, no grid charging, storage power limits, state-of-charge limits, a grid export cap, and chronological state-of-charge carryover. The experiments are organized as a ladder: baseload reference, forecast selection, deterministic rolling-horizon dispatch, uncertainty-aware scenario dispatch, and oracle upper-bound dispatch. The causal ridge model produced the lowest power forecast error, with 21.24 MW RMSE. In deterministic rolling-horizon testing, a 48-hour horizon gave the best causal result among 24, 48, 72, and 168 hours. In the final uncertainty-aware experiment, the best three-scenario 48-hour controller increased revenue by 30.19% and reduced COVE by 23.19% relative to baseload, with zero reported grid and state-of-charge violations. The oracle case, which unrealistically sees the real future, reached a 32.83% COVE improvement and therefore shows the remaining value gap. These results support a practical conclusion: the dispatch breakthrough is not one model by itself, but the combination of causal forecasting, rolling-horizon replanning, uncertainty scenarios, and storage-feasible execution.

Index Terms and Abbreviations

Term	Meaning in this project
Baseload	Reference case where wind is sold directly instead of actively shifted by storage.
COVE	Cost of Valued Energy; in the code it is a fixed cost-like value divided by the realized revenue metric. Lower is better.
Gurobi	The optimization solver used to solve the mixed-integer dispatch problem.
MILP	Mixed-integer linear programming; a linear optimization model with continuous power variables and binary operating-mode variables.
LMP	Locational marginal price in USD/MWh. This is the electricity price

Term	Meaning in this project
	used when scoring delivered energy.
SoC	State of charge, or how much energy is currently stored.
Oracle	A non-realistic upper bound where the optimizer sees actual future wind and price.
Scenario	One possible future path for wind and price. Multiple scenarios let the controller avoid trusting only one forecast.

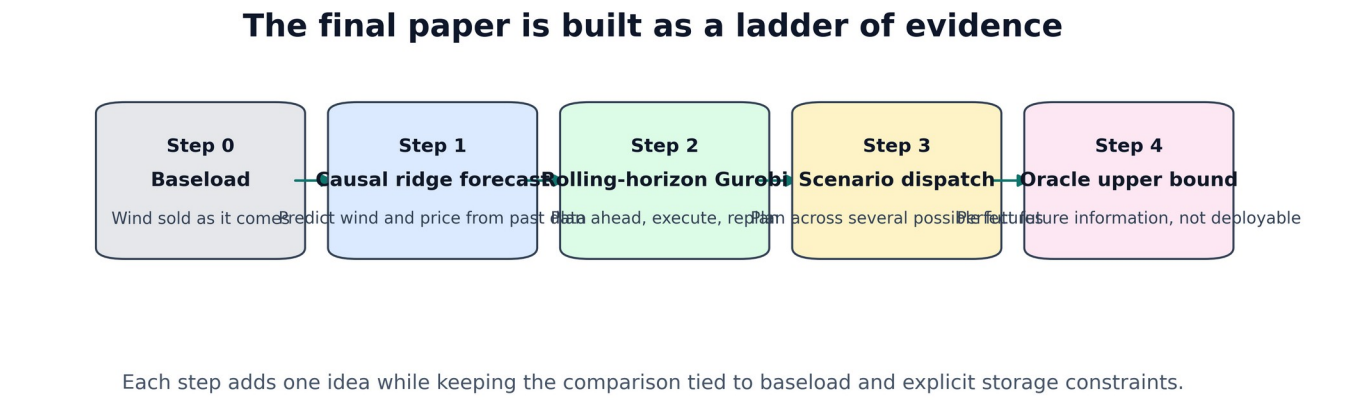

Figure 1. *The project is written as a ladder. Each step adds one new idea while keeping the result tied back to baseload and explicit storage constraints.*

1. Introduction

The central problem is simple to say but hard to solve: a wind farm does not control when the wind blows, and the grid price changes every hour. If wind is strong during a low-price hour, selling it immediately may be less valuable than storing part of it and releasing it later. The catch is that storage decisions must be made before the future is known. A controller needs a forecast, but forecasts are imperfect.

This project studies a hybrid wind-storage system. The wind farm produces power. A storage system can charge from wind, hold energy, and discharge later. The grid receives direct wind plus discharged storage energy. The goal is not only to make revenue high, but to do so while obeying physical rules: the storage cannot charge and discharge at the same time, it cannot overfill, it cannot go below its minimum state of charge, and it cannot send more power to the grid than the export limit.

The paper is intentionally built around a ladder of evidence. Step 1 asks which forecast is best. Step 2 asks which rolling-horizon length makes sense. Step 3 asks whether several possible futures improve the decision compared with one predicted future. Step 4 compares everything against an oracle upper bound. This keeps the main result easy to explain: each new layer is tested against a baseline instead of being mixed into one unexplained black box.

Contribution summary: The contribution is a reproducible forecast-to-dispatch workflow for hybrid wind storage. It combines causal machine learning forecasts, MILP dispatch, scenario uncertainty, and storage-feasible realized execution on a long historical Pyron wind and price test set.

2. Case Study and Data

The case study uses the Pyron wind farm data organized in the project repository. Pyron is treated as a West Texas wind-storage dispatch site connected to ERCOT price information. ERCOT, the Electric Reliability Council of Texas, operates the Texas grid and publishes electricity market price data. The dispatch score uses LMP, which means the dollar value of energy at a specific location and hour.

The long processed dataset in the repository covers 1980 through 2023. The scenario experiment uses the early period to train and calibrate the forecast residuals, then tests on unseen 2014-2023 data. In plain language, the model learns from the past and is judged on later years that were not used for fitting.

The storage assumptions are based on the compressed-air energy storage case discussed with Nora Hosseiniimeni and Dr. Qin, and the PNNL Energy Storage Cost and Performance Database. The final scenario experiment uses 100 MW storage power, 10 hours duration, 1000 MWh energy capacity, 200 MWh minimum SoC, 600 MWh initial SoC, 55% round-trip efficiency, and a 249 MW grid export cap.

Item	Value used in the final scenario run
Storage technology	Compressed-air energy storage style configuration
Power rating	100 MW
Duration	10 hours
Energy capacity	1000 MWh
Minimum SoC	200 MWh
Initial SoC	600 MWh
Round-trip efficiency	55%
Grid export limit	249 MW
Charging source	Wind only
Grid charging	Not allowed
Testing period	2014-2023 frozen test period

3. What Baseload Means

Baseload is the reference case. In this project, it does not mean a coal or nuclear plant. It means the wind farm sells wind directly instead of using a smart storage schedule. Every improvement number is compared against this reference.

This is why baseload is important. A complicated controller is not automatically useful. If the forecast is bad and the storage schedule is wrong, the controller can store energy during the wrong hours or release it too early. The project therefore asks whether each added method actually beats the simple reference case.

4. Part 1: Causal Ridge Forecasting

The first machine learning step predicts wind power. The final selected forecast is a causal ridge regression. It is causal because it only uses information that would already be known at the time of prediction. It does not peek at future wind or future price.

Ridge regression is a regularized linear model. Regularization means the model is discouraged from making extremely large coefficient choices just to fit old noise. This matters because wind data is noisy, and a model that memorizes old noise may fail on new years.

The feature list from the code includes current wind speed, speed squared, speed cubed, lagged power from 1, 2, 3, and 24 hours earlier, and time-of-day/year signals. Speed squared and speed cubed help because turbine power is strongly nonlinear in wind speed; small wind-speed changes can create much larger power changes when the turbine is in its active power-producing region.

```
FEATURE_NAMES = [
    "bias", "speed", "speed_sq", "speed_cu",
    "lag_power_1h", "lag_power_2h", "lag_power_3h", "lag_power_24h",
    "hour_sin", "hour_cos", "day_sin", "day_cos",
]

weights = solve((X.T @ X + alpha * I), X.T @ y)
forecast_power = clip(X @ weights, 0, rated_power)
```

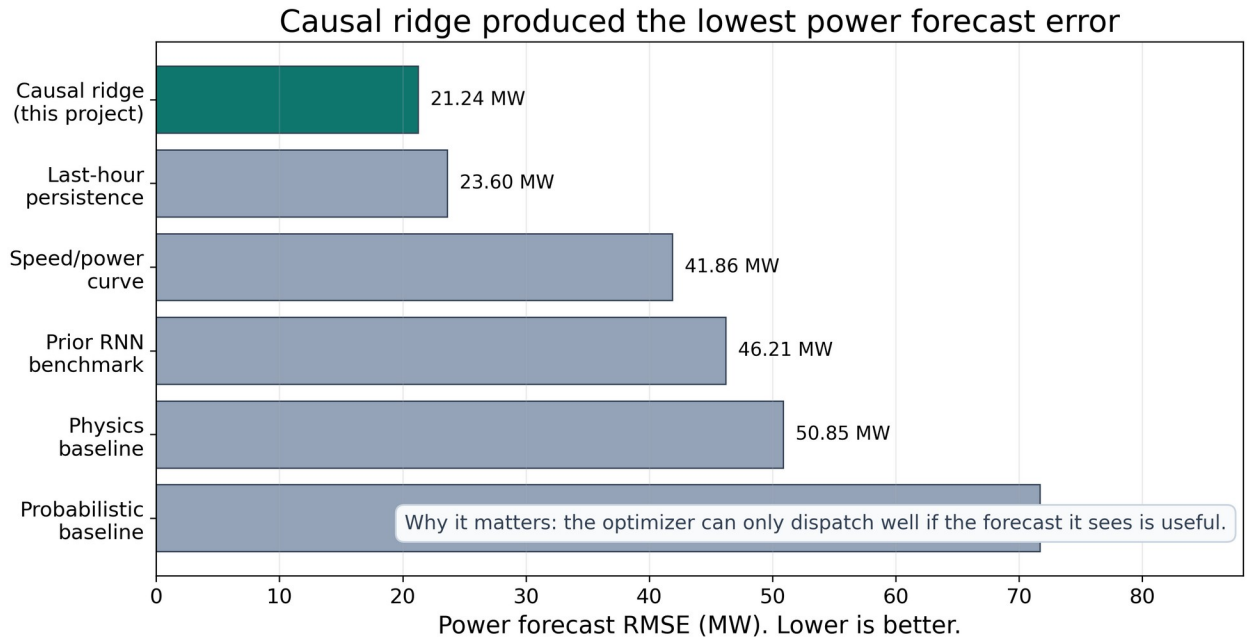

Figure 2. *Forecast comparison for Part 1. Lower RMSE means the predicted power is closer to the measured power. The causal ridge forecast had the lowest RMSE at 21.24 MW.*

The forecast comparison matters because Gurobi can only optimize the information it receives. If the wind forecast is wrong, Gurobi may make a mathematically optimal plan for the wrong future. The causal ridge forecast was selected because it had the lowest RMSE among the tested forecast options.

Forecast method	What it means	RMSE (MW)	Role in the paper
Causal ridge	Regularized ML model using wind speed, lagged power, and time features.	21.24	Selected forecast
Last-hour persistence	Assumes the next value looks like the last measured value.	23.60	Simple baseline
Speed/power curve	Uses a fitted wind-speed-to-	41.86	Physics-inspired baseline

Forecast method	What it means	RMSE (MW)	Role in the paper
	power curve.		
Prior RNN output	Saved recurrent neural network benchmark from earlier work.	46.21	Reference comparison
Physics baseline	Saved physics-style prediction from earlier work.	50.85	Reference comparison
Probabilistic baseline	Saved probabilistic prediction from earlier work.	71.69	Reference comparison

5. Part 2: MILP Dispatch With Gurobi

After predicting wind and price, the second part decides what to do with the energy. This is where Gurobi is used. Gurobi solves the MILP dispatch problem. The continuous variables are charge power, discharge power, direct wind sent to the grid, delivered power, and state of charge. The binary variable chooses whether the storage is in charging mode or discharging mode.

The most important idea is that the optimizer is not allowed to invent energy. If the forecast says wind will be available, Gurobi may plan to charge from that wind. During realized execution, the action is checked against actual wind and actual SoC. Any infeasible part is clipped, and remaining wind that cannot be delivered is recorded as curtailment.

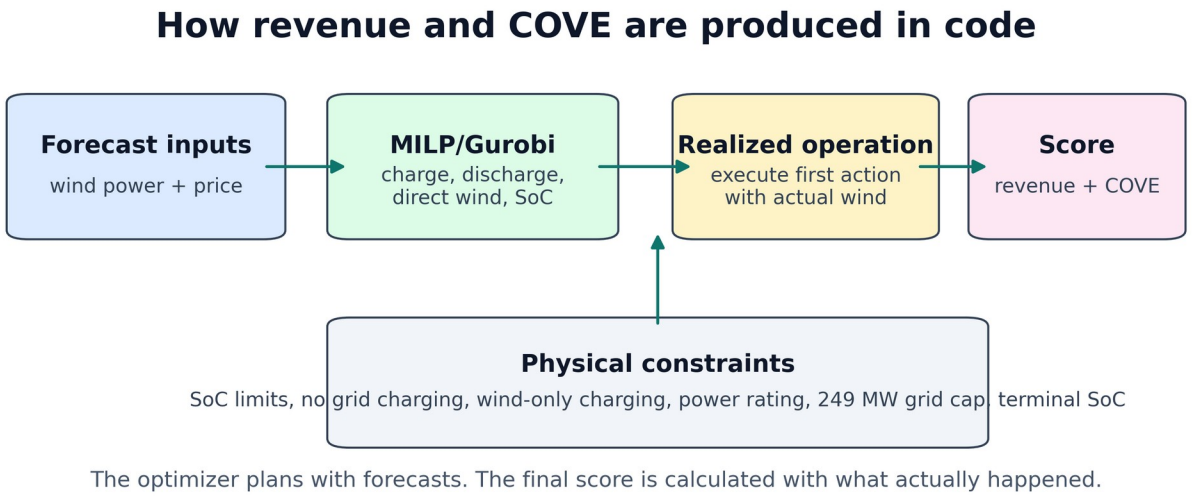

Figure 3. *Code-level flow for revenue and COVE. Forecasts enter the MILP, the first action is executed against actual values, then the realized delivered energy is scored.*

The revenue and COVE calculations are short in code. Revenue is the realized price multiplied by realized delivered power. COVE is the fixed cost-like numerator divided by the valued delivered energy/revenue metric. Since the numerator is fixed inside a given experiment block, lowering COVE is equivalent to increasing the valued energy delivered under that block.

```
for each hour t:
    delivered[t] = direct_wind[t] + discharge[t]
    revenue += raw_realized_lmp[t] * delivered[t]


cove = fixed_cost / revenue
cove_improvement_pct = 100 * (baseload_cove - dispatch_cove) / baseload_cove
```

```
revenue_gain_pct = 100 * (dispatch_revenue - baseload_revenue) / baseload_revenue
```

6. Storage Constraints

The constraints are the reason this is an energy dispatch model rather than just a revenue curve fit. Without constraints, a controller could look good by doing physically impossible things, such as charging and discharging at the same time or discharging energy that is not in storage.

Constraint	Meaning
Cmin <= SoC(t) <= Cmax	The storage level must stay between minimum and maximum capacity.
0 <= P_ch(t) <= P_ES * u(t)	Charging is limited by storage power and by charging mode.
0 <= P_dis(t) <= P_ES * (1 - u(t))	Discharging is limited by storage power and by discharging mode.
P_dis(t) <= available SoC	The model cannot discharge energy that is not stored.
0 <= P_dir(t) <= wind(t)	Direct wind sent to the grid cannot exceed available wind.
P_ch(t) <= wind(t) - P_dir(t)	Storage can charge only from leftover wind, not from the grid.
P_delivered(t) = P_dir(t) + P_dis(t)	Grid delivery equals direct wind plus storage discharge.
0 <= P_delivered(t) <= 249 MW	The grid export cap is enforced.
SoC(t+1) = SoC(t) + charge - discharge/efficiency term	Storage carries forward chronologically hour by hour.
Terminal SoC rule	The planned horizon closes the storage balance, and realized execution carries SoC forward.

This table also explains why the curtailment correction mattered. If actual wind is higher than the forecast, the controller should not automatically sell all extra wind after it has already planned a smaller direct-wind amount. The corrected recourse rule respects the planned direct-wind allocation, actual wind, charging limits, and grid capacity. Extra wind that cannot be used becomes curtailment.

7. Rolling Horizon

A rolling horizon means the optimizer repeatedly looks ahead, chooses an action, executes only the near-term action, updates the battery, and solves again. This is closer to real operation than making one fixed schedule for an entire year.

For example, with a 48-hour horizon, Gurobi sees two forecasted days. It chooses a charge/discharge/direct-wind plan, but the controller only commits the first executed block. After time moves forward, the forecast is updated and Gurobi solves again. This lets the controller correct some forecast mistakes.

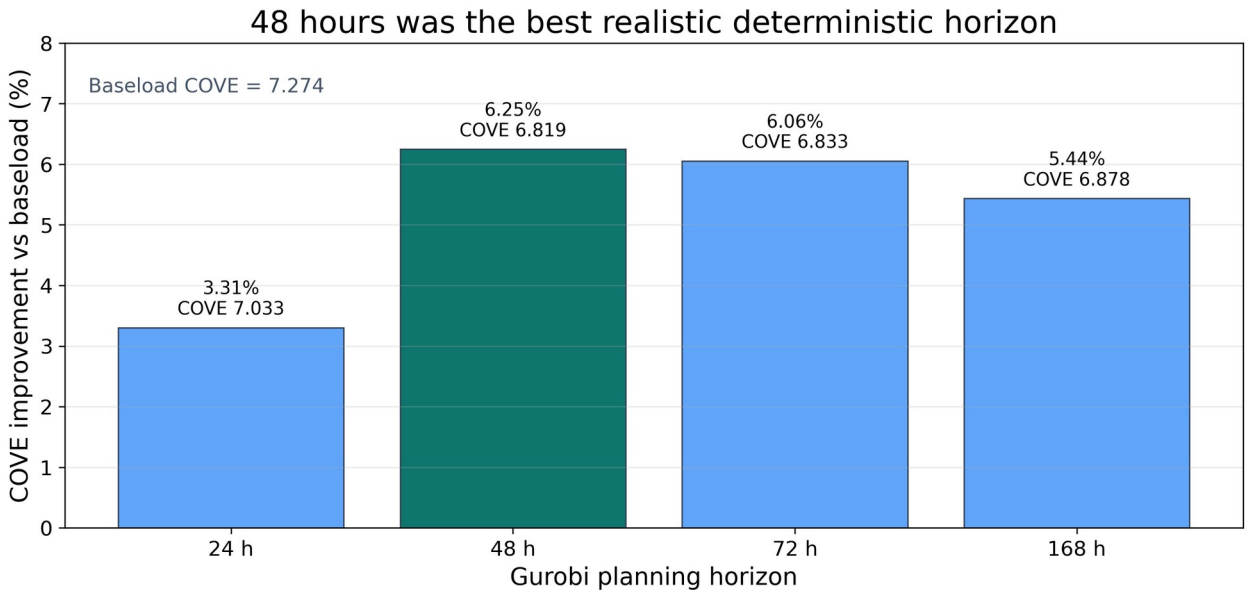

Figure 4. *Deterministic horizon comparison using the causal ridge forecast. The 48-hour horizon gave the highest COVE improvement in this frozen horizon-selection run.*

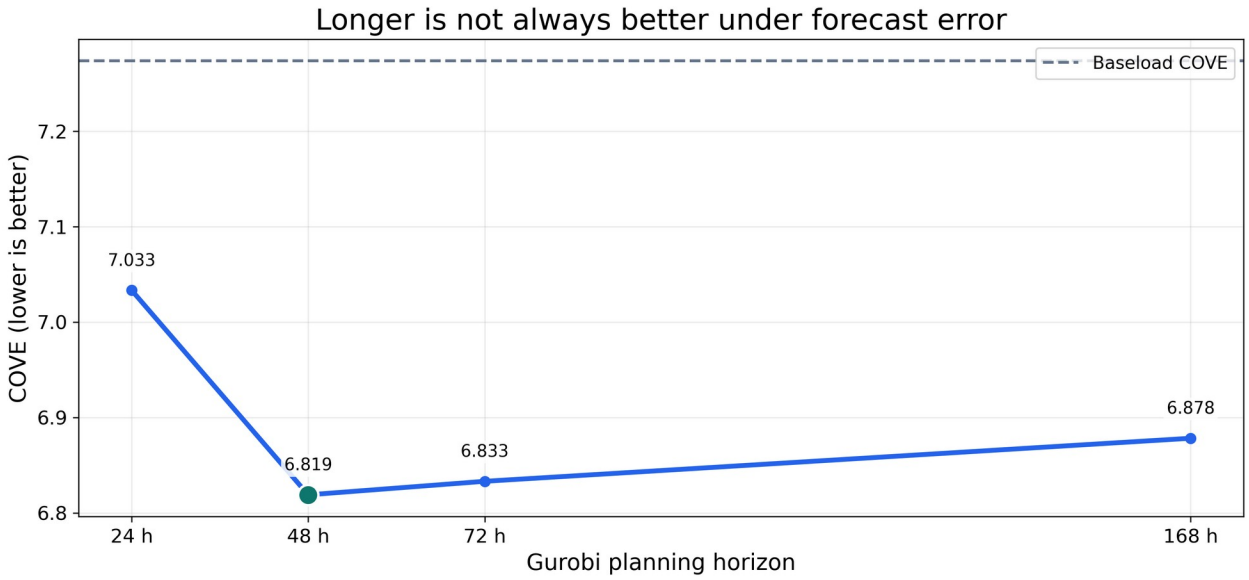

Figure 5. *COVE by deterministic horizon. Lower COVE is better. The line shows that looking farther ahead helps at first, but too long a forecast horizon can add error.*

The 48-hour result is important because it explains the later scenario choice. A 24-hour horizon has limited ability to prepare for tomorrow. A 168-hour horizon sees a whole week, but the forecast is much less reliable that far out. The 48-hour window was the best compromise in the deterministic causal test.

8. Scenario Dispatch

The scenario controller is the main paper contribution. Instead of asking Gurobi to trust one forecast path, it gives Gurobi several possible wind and price futures. The first-hour decision must be the same across scenarios, which is called non-anticipativity. That means the controller cannot secretly choose a different first action for each future before it knows which future will happen.

The scenarios are built from forecast residuals. A residual is the difference between what the forecast predicted and what actually happened. By looking at old residuals, the model creates plausible low, middle, and high future paths. This is why the method is uncertainty-aware: it plans against several possible mistakes, not just the central forecast.

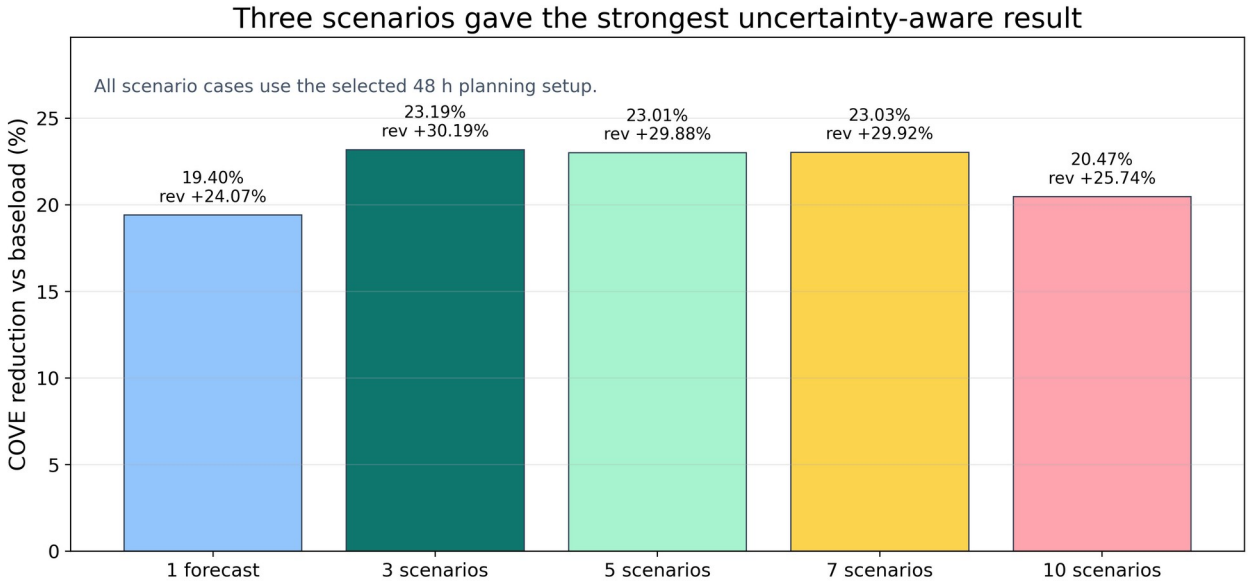

Figure 6. *Scenario count comparison using the selected 48-hour setup. The three-scenario controller had the strongest COVE reduction at 23.19% versus baseload.*

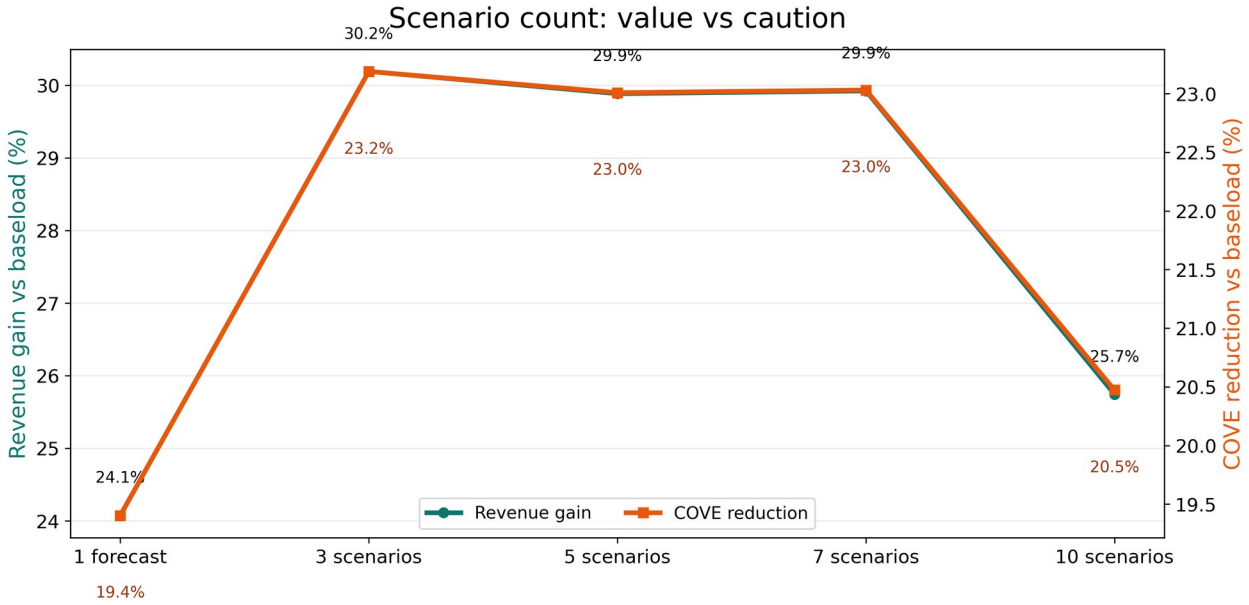

Figure 7. *Revenue gain and COVE reduction by scenario count. More scenarios are not automatically better. Ten scenarios became too conservative and lost value.*

The best scenario result used three scenarios. Five and seven scenarios were very close, but three was slightly better in the frozen full run. Ten scenarios performed worse because the model became too cautious. In simple terms, it protected against too many possible futures and gave up some profitable opportunities.

Case	Revenue	Revenue gain	COVE	COVE reduction
Baseload	$271,870,402.70	0.00%	0.215746	0.00%
Single forecast	$337,322,348.04	24.07%	0.173884	19.40%
3 scenarios	$353,949,333.45	30.19%	0.165716	23.19%
5 scenarios	$353,117,910.43	29.88%	0.166106	23.01%
7 scenarios	$353,220,656.50	29.92%	0.166058	23.03%
10 scenarios	$341,858,797.71	25.74%	0.171577	20.47%

9. Oracle Upper Bound

The oracle is not a real controller. It gives Gurobi the true future wind and price. That is impossible in actual operation, but it is useful because it shows the ceiling: how much value would be available if forecasts were perfect.

The oracle result answers a different question from the deployable scenario result. The scenario controller asks what can be done with imperfect forecasts. The oracle asks how much value is still left on the table because the future is uncertain.

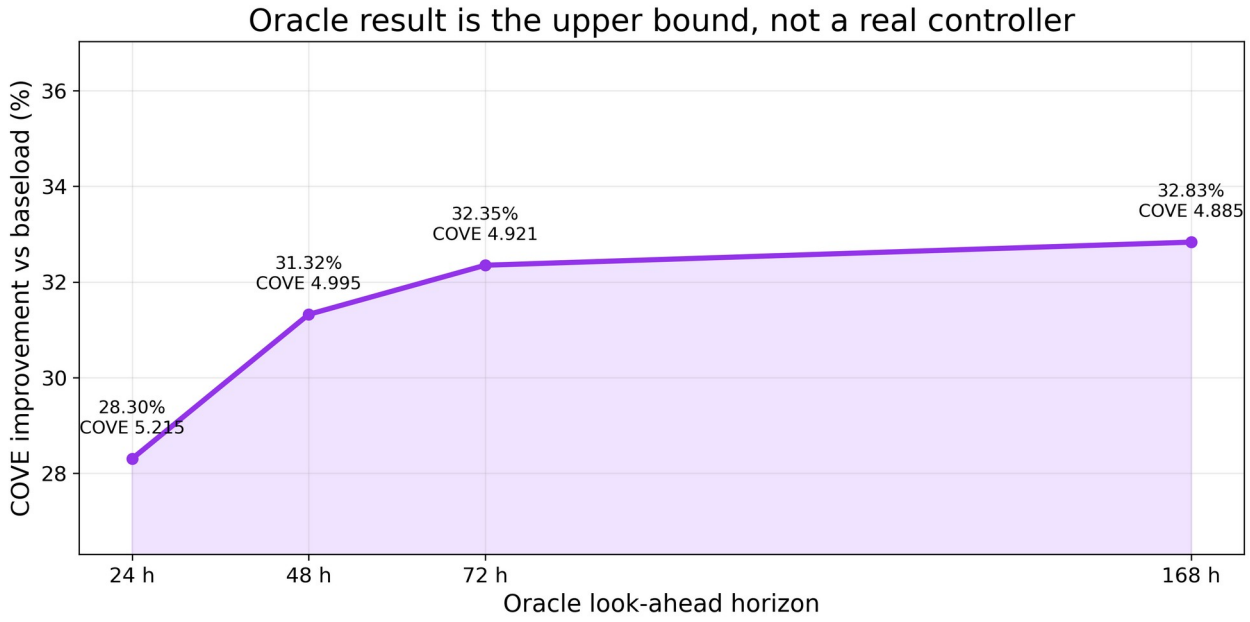

Figure 8. *Oracle upper-bound sweep. The 168-hour oracle reached a 32.83% COVE improvement versus baseload, but it is not deployable because it uses perfect future information.*

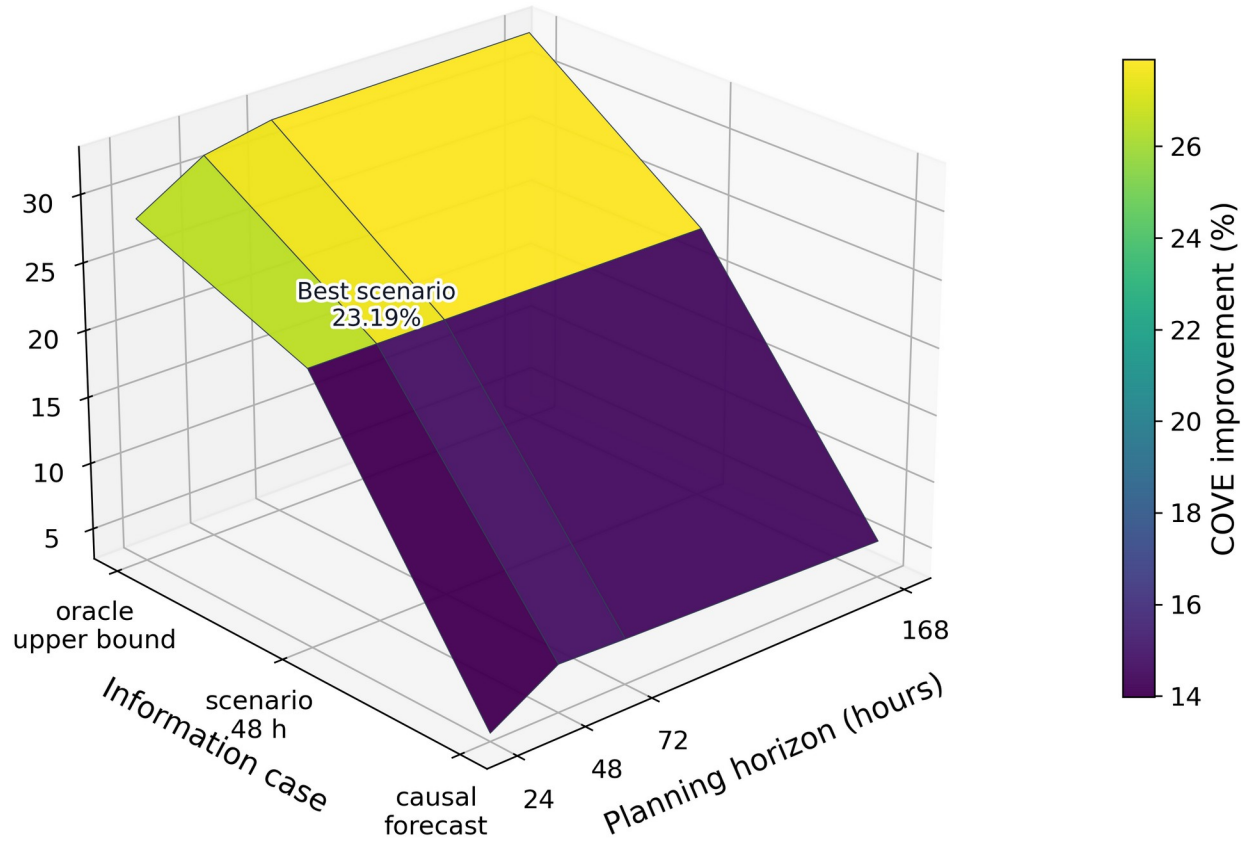

Figure 9. *Information-quality surface. The 3D sheet shows how better information raises the dispatch ceiling. Causal forecasts are lowest, scenarios improve the deployable case, and oracle is the upper bound.*

10. Final Ladder Result

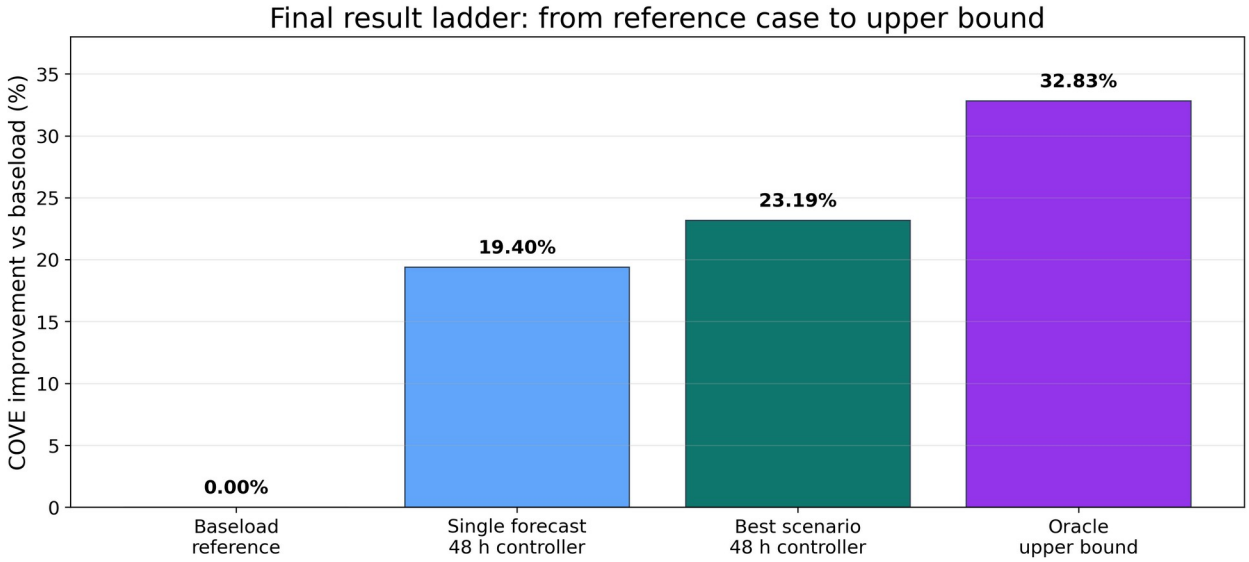

Figure 10. *Final result ladder. The best deployable scenario controller sits between baseload and the oracle upper bound.*

The final ladder should be read carefully. Step 1 does not have COVE because it is only the forecast comparison. Step 2 selects the best deterministic horizon. Step 3 is the final deployable scenario controller. Step 4 is the oracle upper bound. The main result is therefore not that one magic model solved the problem; it is that a realistic controller needs the forecast, the horizon, the uncertainty treatment, and the storage constraints to line up.

11. Why DAM Was Not Used as the Main Result

Day-Ahead Market, or DAM, prices sound attractive because they are known before real-time operation. A DAM experiment was tested as a price-information sensitivity, but it was not used as the headline result. The reason is that the available DAM series did not cleanly match the raw realized PYR_PYRON1 real-time LMP scoring target. Known ahead of time does not automatically mean it is the right signal for this wind farm and price node.

In the tested pipeline, DAM performed worse than the selected causal setup. The likely reason is price mismatch. If the controller plans around a day-ahead price shape that does not match the real-time price used for scoring, it may charge and discharge at the wrong hours. The paper therefore keeps DAM as a documented sensitivity and not as the main result.

12. Newer Proxy Data

A newer proxy dataset was also assembled from public pieces to explore whether the workflow could be extended beyond the original processed Pyron dataset. This is useful as a sanity check, but it is not treated as the official result because the proxy file is not exactly the same validated Pyron data product. The correct paper claim is that the proxy tests showed similar behavior directionally, while the official numbers come from the frozen repository outputs.

13. B6 Verification

The B6 package is a separate verification task requested by Dr. Qin. It reran six 2020 cases for three architectures under a frozen setup. Its purpose was not to create the main paper result. Its purpose was to prove that the code can produce reproducible hourly CSV outputs with raw realized LMP scoring, corrected SoC indexing, corrected planned-versus-realized direct wind execution, and zero annual terminal SoC violations.

This matters for the paper because it increases trust in the implementation. It shows that the dispatch code can be audited under a narrow test case before being used for broader research claims.

14. Reproducibility

The current repository is organized so a reviewer can rerun the paper-facing ladder from the Summer 2026 REU folder. The commands below are the clean entry points. They regenerate the official printed tables and figures from the saved code and frozen outputs.

Step	Folder	Command	Output
1	Summer 2026 REU/causal ridge regression	../../venv/bin/python RUN_1_FORECAST_RMSE.py	Forecast RMSE table
2	Summer 2026 REU/rolling horizon	../../venv/bin/python RUN_2_ROLLING_HORIZON.py	Horizon COVE/revenue table
3	Summer 2026 REU/different scenarios	../../venv/bin/python RUN_3_SCENARIO_COMPARISON.py	Scenario comparison table
4	Summer 2026 REU/oracle upper bound	../../venv/bin/python RUN_4_ORACLE_UPPER_BO	Oracle upper-bound table

Step	Folder	Command	Output
		UND.py	

The current frozen commit used for the organized REU folder is 11b5214e6c16ea174b09deaee292fb772fdf4163. The broader repository still contains archived and exploratory material, but the paper-facing commands above are the official route for the results reported here.

15. Discussion

The central mechanism is information quality. Gurobi can find the best schedule for the information it is given, but it cannot fix a bad forecast by itself. This is why the oracle improves with longer look-ahead, while the causal controller peaks earlier. The oracle sees the real future, so a longer horizon gives it more useful information. The causal controller sees predicted values, so a longer horizon eventually gives it more wrong information.

The scenario result improves the deployable case because it reduces dependence on one exact future. Instead of asking, 'What if this single forecast is right?', the scenario controller asks, 'What first decision is good across several reasonable futures?' That is closer to real decision-making under uncertainty.

The result also shows that more complexity is not always better. Ten scenarios did not win. This is an important finding because it means uncertainty-aware dispatch needs calibration. Too few scenarios may miss risk, but too many or poorly weighted scenarios can make the controller too conservative.

16. Limitations

The first limitation is that the official results depend on the processed data available in the repository. The newer proxy data is useful but not yet a fully validated replacement. The second limitation is that DAM was tested but not adopted because it did not match the real-time price scoring target well enough. The third limitation is that the oracle upper bound is not deployable. It is included only to show the value ceiling.

A final limitation is that the deterministic horizon-selection run and the final scenario run should always be reported with their storage assumptions. The final scenario result is the main deployable claim, while the deterministic horizon sweep is used to explain why 48 hours was chosen. Keeping those roles clear prevents mismatched comparisons.

17. Conclusion

This project developed a complete forecast-to-dispatch workflow for hybrid wind storage. The forecast layer selected a causal ridge model because it had the lowest power RMSE. The dispatch layer used Gurobi to enforce storage constraints and choose charge, discharge, and direct-wind actions. The rolling-horizon layer showed that a 48-hour planning window was the strongest deterministic causal choice. The uncertainty layer showed that a three-scenario controller improved the final deployable result, increasing revenue by 30.19% and reducing COVE by 23.19% relative to baseload. The oracle upper bound showed that perfect information could reduce COVE by 32.83%, leaving a clear but realistic gap between deployable forecasting and perfect hindsight.

The main conclusion is practical: a hybrid wind farm should not be evaluated using forecasting alone or optimization alone. The useful system is the full pipeline: causal forecasts, rolling-horizon replanning, uncertainty-aware scenarios, and physical storage constraints.

Acknowledgments and AI Disclosure

The author thanks Dr. Chris Qin, Nora Hosseiniimeni, Zach Lawrence, and Jessica Yao for guidance, project context, repository materials, and feedback. Generative AI tools assisted with code organization, writing support, figure creation, document formatting, and explanation. Numerical results reported in this manuscript were generated by local repository scripts and reviewed against saved CSV outputs.

References

[1] Pacific Northwest National Laboratory, Energy Storage Cost and Performance Database. https://www.pnnl.gov/projects/esgc-cost-performance

[2] Electric Reliability Council of Texas, Market data and price data products. https://www.ercot.com/mp/data-products

[3] Gurobi Optimization, Gurobi Optimizer documentation and product information. https://www.gurobi.com/solutions/gurobi-optimizer/

[4] Z. Lawrence and J. Yao, prior deep-wind model-dispatch project materials and repository code used as project foundation. https://github.com/davidvalenta-dev/deep-wind-model-dispatch

[5] N. Hosseiniimeni and C. Qin, energy storage dispatch formulation and ASME project materials shared with the author.

[6] IEEE, IEEE BigData conference author information and publication context. https://bigdataieee.org/

Appendix A: Exact Paper-Facing Files

Result	Repository file
Forecast comparison	Summer 2026 REU/causal ridge regression/results/forecast_model_rmse_comparison.csv
Deterministic horizon summary	Summer 2026 REU/rolling horizon/results/causal_ridge_rolling_horizon_summary.csv
Scenario summary	Summer 2026 REU/different scenarios/results/scenario_48h_full_ladder/uncertainty_aware_summary.csv
Oracle upper-bound summary	Summer 2026 REU/oracle upper bound/results/oracle_upper_bound_summary.csv
B6 verification results	Summer 2026 REU/b6 verification/b6_final_results/